ÁvilaOS

Proyecto 1 - Sistemas Operativos

Autor: Elio Colmenares. Desarrollo individual. **Fecha de esta versión:** 29 de octubre de 2026. **Entorno:** Java 25, Apache NetBeans, Maven y Swing. Gson 2.14.0 se limita a JSON streaming; el gráfico usa Java2D.

Propósito

Simular un sistema operativo distribuido con al menos dos computadores. Cada computador reúne CPU, RAM acotada, controlador de E/S, núcleo y una cola de listos. Los procesos se asignan manualmente; no migran entre máquinas. Se implementan FCFS, EDF, Round Robin y prioridades apropiativas, productor-consumidor local/remoto, persistencia y métricas.

Organización y alcance

La GUI observa instantáneas y envía órdenes; no decide la planificación. El coordinador avanza un ciclo lógico por vez y cada CPU ejecuta como máximo una instrucción en ese ciclo. Los buffers ocupan RAM del anfitrión. Disco, memoria virtual y suspensión pertenecen al proyecto posterior.

Este informe explica las clases y decisiones principales, presenta dos UML y compara resultados medidos con cargas equivalentes. No pretende documentar cada línea de código. Las guías por módulo permiten ampliar cada tema; el resumen de defensa reúne lo esencial en dos páginas.

Estado verificable

La compilación limpia repetida el 29 de octubre pasa **25550 comprobaciones**. El estudiante confirmó un build previo de 25454 comprobaciones y la ejecución visual local: ciclo 12, dos completados, CPU global 50 %, respuesta 0.5 y espera promedio de semáforo 6 ciclos. Las 96 comprobaciones posteriores verifican los experimentos comparativos de este informe.

Los experimentos provienen de `FinalAcceptanceTest` y `docs/verification/data/final-comparisons.json`. Los valores del informe se leen de ese archivo generado por el motor. Las pruebas manuales restantes y GitHub se identifican en la sección final; este documento no constituye envío académico.

Recorrido de lectura

Arquitectura y API; ciclo/planificación/memoria; buffers y persistencia; métricas; resultados de políticas; resultados local/remoto; UML de clases y de estados; reproducción y cierre de entrega.

1. Arquitectura y API principal

Los paquetes separan estructuras, procesos, hardware, núcleo, planificación, sincronización, simulación, configuración, métricas e interfaz. Las estructuras de almacenamiento y el ordenamiento son propios.

Clases principales	Responsabilidad	Métodos clave
Computer / Kernel	Ensambla componentes; admisión, estados, despacho y terminación.	onTick; submit; configureScheduling
Cpu / Ram / IoController	Ejecutar una instrucción; reservar memoria; finalizar E/S por ciclos.	onTick; reserve / release; request / pollCompleted
Scheduler / SchedulingPolicy	Una cola de listos y estrategia intercambiable.	select; reorder; shouldPreempt
ProcessDefinition / ProcessControlBlock	Atributos iniciales y estado vivo/contexto del proceso.	admit; dispatch; executeInstruction; block; terminate
Simulation / GlobalClock / ComputerWorker	Propiedad del clúster, órdenes y confirmación de cada ciclo.	advanceCycle; publishState; executeCycle; step
BufferService / BoundedBuffer / SimulatedSemaphore	Protocolo local/remoto, contenido y permisos/esperas de PCB.	prepare; afterInstruction; cleanup; waitFor; signal
SimulationConfig / ScenarioJson / JsonConfigurationStore	Validación común, tokens JSON y archivos UTF-8 seguros.	read / write; load / save
MetricsCalculator / MetricsSnapshot	Calcular por máquina y agregar bases de ciclos confirmados.	computer; aggregate; calculate
UiSession / UiController	Serializar acciones/cierre fuera del EDT; presentar resultados.	loadScenario; saveScenario; refresh
DashboardPanel / UtilizationChart	Ensamblar vistas y dibujar historial conservado por el motor.	render; setSnapshot; paintComponent

Enums e interfaces

Contrato/tipo	Uso
Tickable	onTick(long cycle): Computer, Cpu e IoController reciben un ciclo.
ClockDriver	Simulation implementa advanceCycle, publishState y shutdown para GlobalClock.
SchedulingPolicy	Contrato de selección/reordenamiento/apropiación de las cuatro políticas.
ConfigurationStore	load(Path) y save(Path,SimulationConfig), separadas de la sesión.
Enums principales	ProcessState, ProcessType, BlockReason, CpuMode, SchedulingPolicyType, TerminationReason; InstructionKind y BufferPhase describen las instrucciones/fases.

Los tipos fijos se representan mediante enums. Las interfaces aíslan comportamientos intercambiables o contratos de coordinación. Ningún componente de hardware consulta una ventana para decidir su comportamiento.

2. Ciclo global, planificación y memoria

Un ciclo completo

• El coordinador toma un lote de órdenes pendientes. Las altas, cambios de política/quantum y duración se aplican entre ciclos.

• BufferService entrega despertares cuya disponibilidad ya llegó, sin llamar al Kernel mientras mantiene su mutex.

• Cada ComputerWorker recibe el ciclo y confirma su trabajo. Se recorre en orden estable de ID: los hilos son distintos, pero el orden lógico de trabajo se coordina para obtener trazas reproducibles.

• El núcleo procesa E/S completada, admite según RAM, reordena listos, decide apropiación y despacha. Una CPU ejecuta como máximo una instrucción.

• Se procesan bloqueo, terminación y deadlines. La admisión al cierre no concede una segunda instrucción en ese mismo ciclo.

• Solo después de confirmar todos los computadores se calculan métricas e historial y se publica el siguiente límite global. Un fallo parcial no se cuenta como ciclo completado.

Cuatro políticas

FCFS: conserva el orden de entrada a listos, sin apropiación de un proceso que aún puede ejecutar. **EDF:** prioriza el deadline absoluto más próximo y puede apropiar. **Round Robin:** rota al agotar su quantum cuando existe otro listo. **Prioridades:** el menor número representa mayor prioridad; permite apropiación. Los empates conservan el orden estable de entrada a listos.

Cambiar de política no reconstruye los PCB ni pierde su contexto. La duración en milisegundos controla el ritmo visible, no las fórmulas en ciclos lógicos. Todas las máquinas utilizan Computer; no se duplica una clase por ID.

Memoria y deadlines

Ram reserva unidades y MemoryAdmission relaciona cada reserva con su PCB. Un proceso que no cabe aún permanece NEW; no se exige que toda la carga quepa simultáneamente. Cada buffer reserva su capacidad completa en la RAM anfitriona. Terminar libera la reserva de proceso y cancela recursos pendientes.

El plazo es relativo a la llegada: deadline absoluto = llegada + plazo. Llegar en ciclo 5 con plazo 1 permite ejecutar en 5 y vence al cierre de ese ciclo si no termina. El bloqueo no detiene el reloj. PC y MAR avanzan al ejecutar una instrucción del proceso; permanecen constantes mientras está bloqueado o esperando CPU.

3. Productores, consumidores y persistencia

Buffer acotado y semáforos

Cada buffer tiene vacíos, llenos y mutex. El productor solicita vacíos antes de mutex, inserta, libera mutex y señala llenos. El consumidor solicita llenos antes de mutex, extrae, libera mutex y señala vacíos. El trabajo de cómputo se intercala según el programa del tipo de proceso.

Cada wait, signal, inserción o extracción ocupa una instrucción en modo núcleo. Un wait bloqueante ya consumió su instrucción: al despertar no debe repetirse. Los elementos conservan orden FIFO y origen. Una operación registra permisos, mutex y modificación confirmada para poder limpiar correctamente al vencer el plazo.

Los semáforos Java coordinan Threads y protegen datos reales. Los SimulatedSemaphore administran contadores y colas de PCB; bloquear un proceso simulado no estaciona indefinidamente su hilo ComputerWorker. El reloj y las otras CPU siguen avanzando.

Red y limpieza

Un acceso a buffer remoto espera la latencia configurada por elemento antes de adquirir permisos. No ejecuta instrucciones durante la red ni mantiene el mutex del buffer. Una concesión producida en ciclo k despierta en k+1. Al vencer un deadline se cancelan esperas y se devuelven reservas pendientes; una inserción o extracción ya confirmada no se revierte. Este mantenimiento no añade instrucciones ficticias al proceso terminado.

Escenario inicial JSON

ScenarioJson usa tokens de Gson en modo estricto: versión 1, campos obligatorios, enteros exactos, enums y referencias válidas. SimulationConfig aplica las reglas del modelo y de RAM. No usa DOM de Gson ni colecciones para representar el simulador.

Guardar serializa el escenario inicial aplicado. Incluye cambios hechos antes de empezar, excluye borradores y cambios vivos posteriores. No es un checkpoint de PC, colas o semáforos. Cargar valida todo antes de sustituir el motor y queda pausado en cero. Una entrada rechazada conserva el motor anterior.

JsonConfigurationStore escribe UTF-8 en un temporal del mismo directorio, cierra y solicita reemplazo atómico. Si falla, se informa el error; no se elimina primero el destino ni se usa fallback no atómico. No se promete durabilidad ante un corte eléctrico. UiSession realiza E/S y cierre fuera del EDT; Swing conserva capacidad de responder.

4. Métricas y sus denominadores

Indicador	Fórmula/población
Throughput	Completados normales / ciclos transcurridos; vencidos aparte.
CPU	(usuario + núcleo) / ciclos disponibles. Global: computadores × ciclos.
Respuesta	Suma(primera ejecución - llegada) / procesos que recibieron CPU.
Cumplimiento	Completados / (completados + vencidos); pendientes aparte.
Jain	Suma(servicio)² / [admitidos × suma(servicio²)]. Incluir admitidos sin servicio y terminados; excluir vencidos NEW.
Espera de semáforo	Ciclos de episodios cerrados / episodios cerrados, incluidos deadlines. Abiertos aparte; excluir IO/red.

Los indicadores globales se recalculan sumando bases. No se suman porcentajes ni se promedian respuestas por máquina sin ponderar sus poblaciones. Un denominador inexistente se muestra como **Sin datos**, en lugar de inventar un cero. Las esperas abiertas conservan su contador/tiempo aparte del promedio de episodios cerrados.

Ejemplo local comprobado

Consumidor C y productor P en computador 1, buffer B1 de capacidad 1, instrucciones 6 por proceso, RAM 2, intervalo/cuota 1 y plazo 100. Computador 2 permanece inactivo. Tras doce pasos: servicio usuario 2 y núcleo 10; dos completados, ningún vencido; respuesta de C=0 y P=1, promedio 0.5.

Throughput global = 2/12 = 0.167 procesos/ciclo. CPU global = 12/(2×12) = 50 %; usuario 8.33 % y núcleo 41.67 %. Jain = $12^2/[2\times(6^2+6^2)]$ = 1. El buffer produjo y consumió un elemento, está vacío y sus valores vacíos/llenos/mutex son 1/0/1. Una espera cerrada de seis ciclos da promedio 6.

El gráfico conserva una muestra de utilización acumulada por computador y ciclo confirmado. Su eje X usa el número de ciclos completados; la primera muestra es 1. El Timer puede saltar refrescos sin descartar muestras. Reiniciar o cargar crea otra generación e historial vacío.

5. Comparación de políticas

Condiciones iguales

Dos computadores con RAM 64, misma política por experimento, quantum 2, llegada 0, ciclo 500 ms y horizonte fijo de 12 ciclos. Todos los procesos son CPU_BOUND y requieren RAM 4. Solo cambia la política; la asignación y las definiciones permanecen iguales.

Proceso	Computador	Instrucciones	Plazo relativo	Prioridad
A	1	8	9	2
B	1	2	3	0
C	1	4	7	1
D	2	6	20	1

Política	Comp.	Venc.	Throughput	CPU (%)	Cumpl. (%)
FCFS	2	2	0.167	58.33	50.00
EDF	3	1	0.250	62.50	75.00
Round Robin	1	3	0.083	62.50	25.00
Prioridades	3	1	0.250	62.50	75.00

Política	Respondidos / total	Respuesta (ciclos)	Jain
FCFS	2 / 4	0.000	0.490
EDF	4 / 4	2.000	0.865
Round Robin	4 / 4	1.250	0.731
Prioridades	4 / 4	2.000	0.865

Lectura de los resultados

EDF y prioridades coinciden en esta carga porque los deadlines y prioridades ordenan los procesos de la misma forma: B, C y A en computador 1. Completan más procesos de plazo corto; A vence después de recibir parte del servicio. No se concluye que sean universalmente superiores.

FCFS permite terminar A, pero B y C vencen sin recibir CPU. Su respuesta promedio 0 incluye solo a A y D: **respondieron 2 de 4 procesos**. Ese cero no indica mejor atención general. Round Robin reparte servicio con quantum 2, pero los deadlines cortos de B/C y el de A provocan tres vencimientos. Cambiar quantum o carga podría cambiar la comparación; no se extrapola a escenarios no medidos.

6. Comparación local y remota

Dos computadores con RAM 64, FCFS, B1 en anfitrión 1 y capacidad 1, latencia global 2, plazo 100 y participantes de seis instrucciones, RAM 2 e intervalo/cuota 1. La carga es igual dentro de cada pareja de experimentos; cambia únicamente la ubicación del productor.

Caso	Horizonte	Fin de todos	CPU (%)	Resp.	Espera sem.
P local	8	6	37.50	0.000	Sin datos
P remoto	8	8	37.50	0.000	Sin datos
C/P local	16	12	37.50	0.500	6.000
C/P remoto	16	13	37.50	0.000	7.000

Horizonte, finalización, respuesta y espera se expresan en ciclos lógicos. Fin de todos coincide aquí con terminación normal: no hubo vencidos. P = productor aislado; C/P = consumidor y productor.

Qué explica la diferencia

En el productor aislado, el caso local termina en 6 y el remoto en 8: la red añade dos ciclos sin aumentar el trabajo de seis instrucciones. Al medir ambos hasta ciclo 8, el local queda inactivo los dos ciclos finales; ambos tienen CPU global 37.5 %. Por eso es necesario distinguir utilización en un horizonte común de tiempo hasta finalizar.

Con C/P, el consumidor permanece en computador 1 y el productor pasa de 1 a 2. La pareja local termina en 12; la remota en 13. El segundo computador permite que P empiece desde ciclo 0, mientras en el caso local C ejecuta primero. Ese solapamiento compensa parte del retraso: no sería correcto atribuir toda la diferencia observada únicamente a la latencia.

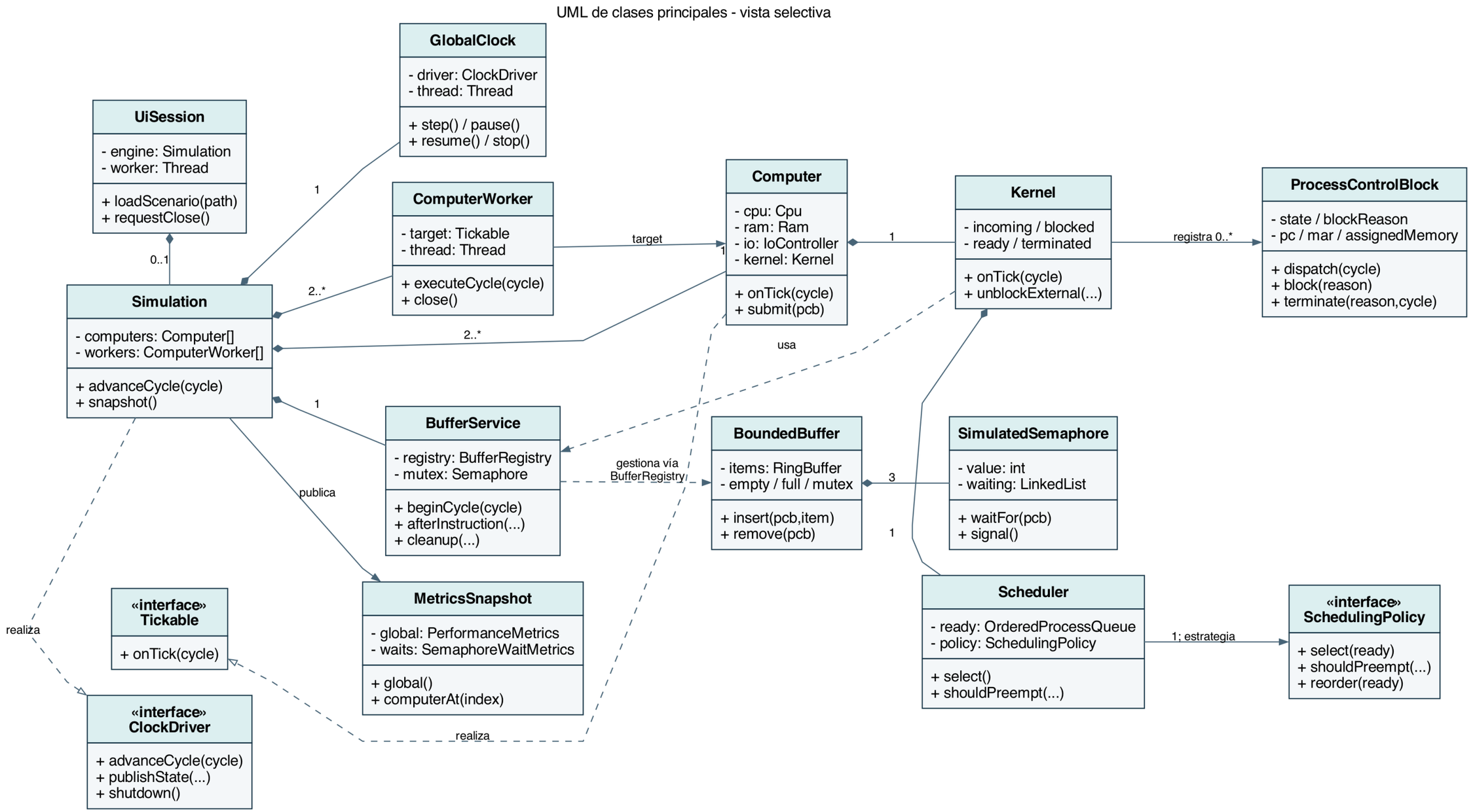
A horizonte común 16, ambas parejas completan dos procesos, producen/consumen 1/1 y usan doce instrucciones: CPU global 37.5 %. La espera cerrada del consumidor pasa de 6 a 7 ciclos; la espera de red del productor no se mezcla con el promedio de semáforos. La respuesta promedio remota 0 refleja que ambos reciben CPU en 0, no que no exista espera posterior.

Conclusión experimental

La política cambia el orden y la población atendida; la red cambia cuándo puede continuar una operación. Los resultados deben leerse con su carga, horizonte y denominador. La arquitectura conserva la misma definición de proceso y permite explicar estas diferencias con trazas, sin alterar el algoritmo para favorecer una política.

7. UML de clases principales

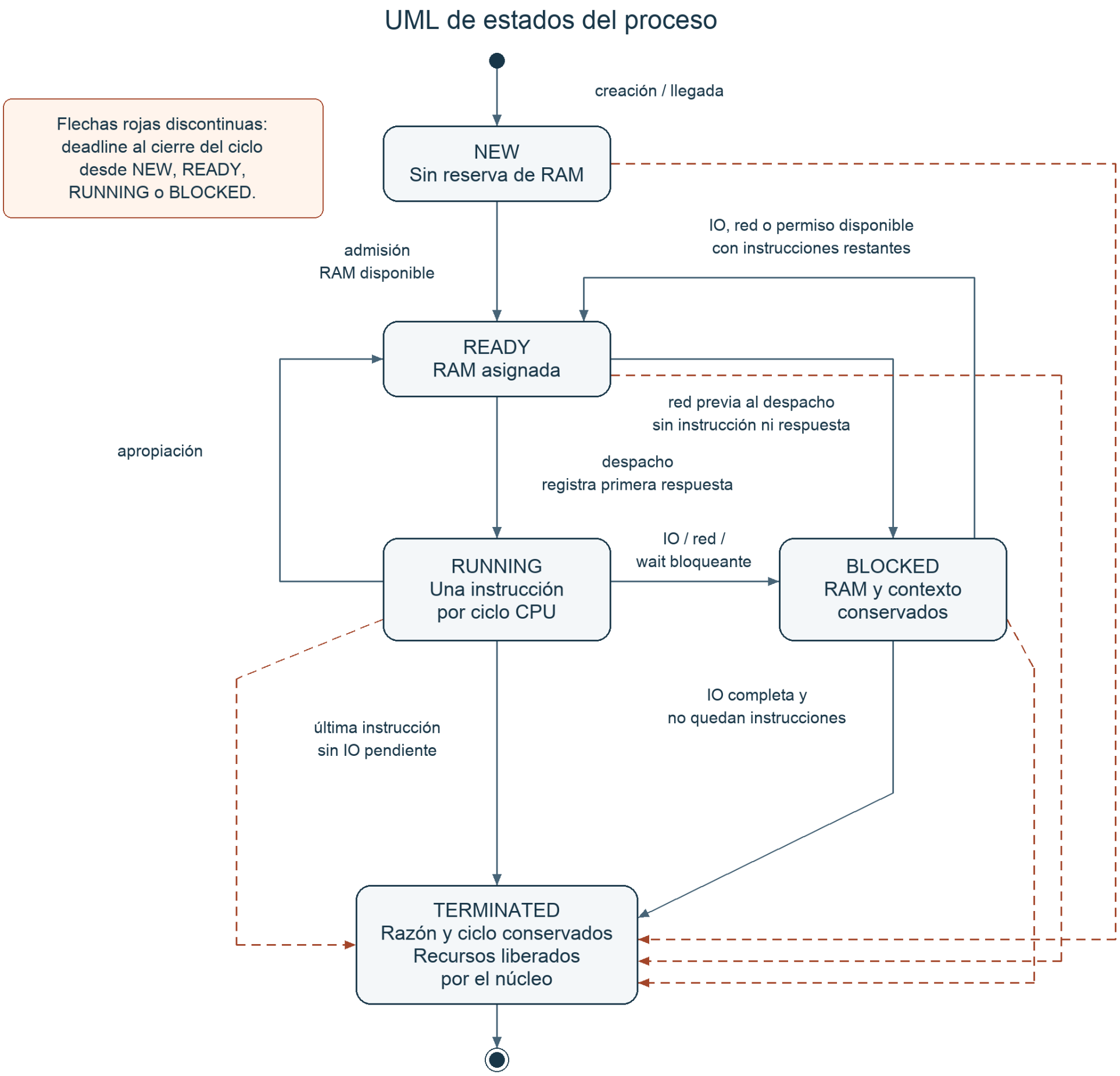
Vista selectiva: composición en rombo relleno; asociación dirigida; dependencia discontinua; realización con triángulo vacío. Computer contiene CPU/RAM/IO/Kernel. El registro de buffers se abstrae en la relación «gestiona vía BufferRegistry». Las políticas concretas son FcfsPolicy, EdfPolicy, RoundRobinPolicy y PreemptivePriorityPolicy.



Esta página usa un formato amplio para conservar legibilidad. Las fuentes y SVG ampliables se incluyen en docs/uml/.

8. UML de estados de proceso

El deadline puede terminar un proceso desde NEW, READY, RUNNING o BLOCKED. Despertar conserva PC/MAR y reserva de RAM. Preparar red antes del despacho no registra primera respuesta; una IO final puede completar la terminación sin ejecutar otra instrucción.



El bloqueo conserva PC/MAR y RAM; solo una instrucción ejecutada avanza PC/MAR.

El núcleo realiza la liberación de RAM, la cancelación de IO y la limpieza de reservas del buffer. TERMINATED conserva la razón, ciclo final y contadores de servicio; no vuelve a listos. Esta vista se genera desde una sola tabla de transiciones con layout fijo para evitar cruces de etiquetas.

9. Verificación, reproducción y entrega

Evidencia disponible

La suite actual tiene **25550 comprobaciones** y ejecuta el motor real. Incluye estructuras/orden estable, políticas, admisión/RAM, deadlines en diversos estados y fases, hilos/coordinación/cierre, productor-consumidor/red, JSON estricto/guardado seguro, sesiones Swing, métricas y las comparaciones de este informe. Son assertions, incluidas invariantes repetidas; no 25550 casos independientes.

El estudiante aportó build de 25454 comprobaciones y capturas de la ventana real en ciclo 12. Confirmó los contadores locales y el promedio de espera 6. Se conservan nombres y hashes de esas capturas en el registro de aceptación, sin publicar el resto de su escritorio.

Ejecución para el corrector

• Instalar JDK 25 y Apache NetBeans. Abrir la carpeta que contiene pom.xml y seleccionar JDK 25 como plataforma del proyecto.

• Clean and Build ejecuta AllTests desde exec (core-tests). Esperar OK: N comprobaciones y BUILD SUCCESS; el bloque previo Surefire puede mostrar cero.

• Run Project abre ÁvilaOS en una JVM separada. Los ejemplos JSON están en examples/. Para repetir el local, cargar local-producer-consumer.json y dar doce pasos.

• La primera construcción necesita acceso a Maven Central para plugins y Gson. No requiere la caché privada del desarrollador ni rutas de su Mac. El README contiene la alternativa con Maven y JDK 25 seleccionados.

Lo que todavía requiere cierre

Guardar/recargar/sobrescribir y rechazar JSON corrupto en ventana real, cerrar la ejecución actual, Debug y abrir una copia limpia en el IDE permanecen en el checklist. Repositorio privado: https://github.com/elioaca01/proyecto-sistemasop. Historial y 12 ramas publicados por SSH; la versión completa está en release/final-delivery. La auditoría del 29 de octubre confirmó 0 Issues y 0 Pull Requests; integrar mediante PR comentados hacia develop/main, comprobar acceso de correctores y efectuar el envío académico siguen pendientes.

Fuentes y artefactos

Enunciado Proyecto 1, Sistemas Operativos 2627-1, suministrado por el estudiante; diseño aprobado del 18 de octubre; código Java en src/main/java; ejecutor AllTests y FinalAcceptanceTest; mediciones en docs/verification/data/final-comparisons.json; evidencia de módulos en docs/verification; guías en docs/defensa. El informe y el resumen se regeneran desde docs/informe/build_report.py.

Conclusión

Se dispone de un simulador por capas con cuatro políticas, memoria acotada, sincronización local/remota, persistencia y observación comprobable. Las pruebas y mediciones sostienen el comportamiento descrito. Completar los pendientes manuales y de repositorio es necesario antes de afirmar que la entrega académica está cerrada.